CompositeCurves Architecture

This document is for contributors who want to understand how `CompositeCurves` is structured, why it is structured this way, and where new features should be added.

Goals

The package is built around a few non-negotiable goals:

- Author piecewise curves in Unity with `ScriptableObject` assets.
- Keep runtime evaluation simple and efficient.
- Support common built-in curve presets.
- Allow user-defined curve formulas with configurable variables.
- Avoid runtime interpretation of custom formulas.
- Keep generated code visible as normal project source.
- Avoid automatic regeneration that causes unnecessary Unity domain reloads.

These goals drive the current split between runtime code, editor code, and generated code.

High-Level Layout

The package currently lives under:

- Assets/CompositeCurves/Runtime
- Assets/CompositeCurves/Editor
- Assets/CompositeCurves/Generated

The responsibilities are:

- `Runtime`: data structures and evaluation API used by game code.
- `Editor`: authoring UI and code generation tools.
- `Generated`: emitted C# source for custom curve expressions.

Core Runtime Model

The main runtime asset is CompositeCurveDefinition.cs.

It owns:

- A persistent `curveId`
- An `outsideRangeMode`
- A list of `CompositeCurveSegment`

It exposes:

- `GetValue(float x)`
- `Evaluate(float x)`
- `TryGetValue(float x, out float value)`

Segment Model

Each segment is represented by CompositeCurveSegment.cs.

A segment contains:

- A persistent `segmentId`
- A human-readable `displayName`
- An enabled flag
- A domain range: `startX` to `endX`
- A start and end bound inclusion mode
- A mode: `Preset` or `Custom`
- Either a preset enum or a custom expression string
- A variable array

The segment is the smallest independently evaluable piece of a composite curve.

Evaluation Flow

Runtime evaluation is intentionally narrow:

1. `CompositeCurveDefinition` rebuilds and sorts enabled segments by domain.
2. A binary-search-like lookup finds the segment covering the requested `x`.
3. The selected segment evaluates either:
   - a preset formula using cached coefficients
   - a generated custom evaluator through the registry
4. If no segment contains `x`, `outsideRangeMode` determines fallback behavior.

Why this is efficient

- Preset segments cache frequently used numeric values in `PrepareRuntimeCache()`.
- Segment lookup does not scan every segment in the common case.
- Runtime custom curves do not parse strings or interpret expressions.
- Custom evaluation is a direct call into generated C#.

Preset Curves

Preset curves are defined through `CompositeCurvePreset` in CompositeCurveTypes.cs.

Current presets:

- Constant
- Linear
- Quadratic
- Cubic
- Sine
- Cosine
- Tangent

- QuadraticBezier
- CubicBezier

Each preset has a default variable set supplied by `CompositeCurveSegment.CreateDefaultVariables(...)`.

This is the extension point if you want to add more built-in curve families.

Variables

Variables are represented by `CompositeCurveVariable` in CompositeCurveTypes.cs.

A variable is just:

- `Name`
- `Value`

For presets, the variable names are semantic, for example:

- Linear: `m`, `c`
- Quadratic: `a`, `b`, `c`
- Trig: `amplitude`, `frequency`, `phase`, `offset`

For custom curves, variables are referenced by name in the authored expression, then bound to generated local variables in generated code.

Generated Custom Curves

This is the most important architectural decision in the package.

Custom expressions are not evaluated by parsing strings at runtime. Instead:

1. The user authors a custom expression in the inspector.
2. The editor tool reads all custom segments from all `CompositeCurveDefinition` assets.
3. It emits a C# source file at CompositeCurveGenerated.Evaluator.g.cs.
4. Unity compiles that file as normal project source.
5. Runtime evaluation routes custom segments through `CompositeCurveGeneratedRegistry`.

The registry is defined in CompositeCurveGeneratedRegistry.cs.

This keeps the runtime side decoupled from the exact generated implementation.

Why a registry exists

The runtime layer should not know about editor-only generation details.

The registry allows:

- a safe fallback if no generated evaluator exists yet
- a stable runtime call site
- generated code to register itself in both editor and play mode

Manual-Only Generation

Code generation is deliberately manual.

You must explicitly trigger regeneration through:

- The curve inspector button: `Regenerate Custom Curves`
- The menu item: `Tools/Composite Curves/Regenerate Generated Curves`

This is intentional because automatic regeneration would rewrite `.cs` files whenever curve assets change, which causes avoidable Unity script recompiles and domain reloads.

Contributor rule

Do not reintroduce automatic asset-postprocessor regeneration unless the workflow requirement changes. The current behavior is a deliberate UX/performance tradeoff.

Editor Architecture

The main editor files are:

- CompositeCurveDefinitionEditor.cs
- CompositeCurveCodeGenerator.cs
- CompositeCurveRuntimeTests.cs

Inspector responsibilities

`CompositeCurveDefinitionEditor` is responsible for:

- Rendering the authoring UI
- Adding/removing/duplicating segments
- Initializing new segment ranges from the last segment only at creation time
- Editing variables
- Sorting segments
- Showing validation warnings
- Showing a graph preview with manual pan/zoom and persistent viewport state
- Offering manual regeneration controls

It is not responsible for evaluating custom expressions directly.

Generator responsibilities

`CompositeCurveCodeGenerator` is responsible for:

- Finding all `CompositeCurveDefinition` assets
- Ensuring stable IDs exist
- Normalizing custom expressions

- Mapping user variable names to safe generated identifiers
- Emitting one generated evaluator source file

It should stay deterministic and easy to reason about.

Expression Normalization

The current generator accepts a simple expression string and performs lightweight normalization, for example:

- `sin(...)` -> `Mathf.Sin(...)`
- `pow(...)` -> `Mathf.Pow(...)`
- `pi` -> `Mathf.PI`

It also rejects obviously unsafe statement-like constructs such as:

- `;`
- `{`
- `}`

This is not a full parser. It is a constrained code-generation convenience layer.

Contributor caution

If you extend custom expression support, avoid creeping toward a half-parser with ambiguous behavior. Either:

- keep the expression system intentionally small and predictable
- or replace it with a real parser/AST pipeline

Do not add fragile ad hoc string replacements without considering correctness.

IDs and Stability

Both curves and segments use generated identifiers:

- `curveId`
- `segmentId`

These are important because generated code dispatches by IDs, not by display names or asset paths.

That means:

- Renaming a segment should not break lookup.
- Renaming a curve asset should not break lookup.
- Reordering segments should not break lookup.

Contributors should preserve this stability model.

Outside-Range Behavior

When `x` falls outside all segment domains, behavior is controlled by `CompositeCurveOutsideRangeMode`.

Current modes:

- `ReturnZero`
- `ClampToEdge`
- `ExtrapolateNearestSegment`

If you change this area, be careful about gaps between segments as well as values strictly before the first and after the last segment.

Design Constraints

There are a few constraints contributors should keep in mind:

- No external packages should be introduced.
- Source should remain visible in the Unity project.
- Editor code should stay under `Editor/`.
- Runtime should not depend on editor APIs.
- Custom curves must remain non-interpreted at runtime.
- Generated source should remain readable enough to debug.
- Prefer Unity APIs and C# syntax that are broadly supported across Unity versions, not just newest-editor conveniences.

Common Extension Points

Adding a new preset

You will usually need to update:

- CompositeCurveTypes.cs
- CompositeCurveSegment.cs
- Possibly CompositeCurveDefinitionEditor.cs if the UX needs special handling

Specifically:

- Add the enum value
- Add default variables
- Add cached-value preparation
- Add evaluation logic

Extending custom expressions

You will usually work in:

- CompositeCurveCodeGenerator.cs

Be careful about:

- Generated identifier safety
- String escaping
- Deterministic output
- Syntax rejection behavior

Improving the inspector

You will usually work in:

- CompositeCurveDefinitionEditor.cs

Changes here should not silently change runtime behavior unless that behavior is clearly reflected in the asset data.

Known Limitations

Current limitations include:

- Custom expression support is intentionally lightweight, not a full math language.
- The generated evaluator is rewritten as a single file for all custom segments.
- Validation is useful but not exhaustive.
- Preview uses runtime evaluation, so stale generated code can make custom segments preview as zero until regeneration happens.
- Preview state is intentionally not auto-refit when bounds change. Contributors should preserve manual control unless the workflow requirement changes.
- Edit mode tests currently focus on runtime semantics. Inspector interaction coverage is still mostly manual.

The last point is expected under the manual-generation model.

Recommended Contribution Workflow

1. Understand whether your change is runtime, editor, or generated-code related.
2. Keep runtime hot paths simple.
3. Prefer adding validation in the editor rather than adding defensive runtime overhead everywhere.
4. If changing custom expression behavior, inspect the generated `.g.cs` output directly.
5. Preserve manual generation unless there is a strong reason to change it.
6. Add or update edit mode tests when changing runtime semantics.

Documentation Split

The package now uses three documentation layers:

- README.md for overview and quick start

- USER_MANUAL.md for authoring guidance inside Unity
- ARCHITECTURE.md for contributors and maintainers

File Reference Summary

- Runtime asset: CompositeCurveDefinition.cs
- Runtime segment: CompositeCurveSegment.cs
- Shared types: CompositeCurveTypes.cs
- Registry bridge: CompositeCurveGeneratedRegistry.cs
- Inspector: CompositeCurveDefinitionEditor.cs
- Generator: CompositeCurveCodeGenerator.cs
- Generated evaluator: CompositeCurveGenerated.Evaluator.g.cs